

fn truncate_domain

Michael Shoemate (@Shoeboxam)

May 15, 2026

This proof resides in “contrib” because it has not completed the vetting process.

Proves soundness of `truncate_domain` in `mod.rs` at commit `f5bb719` (outdated¹).

1 Hoare Triple

Precondition

Compiler-verified

None

Caller-verified

Truncation keys are stable row-by-row transformations of the data.

Pseudocode

```
1 def truncate_domain(  
2     domain: DslPlanDomain,  
3     truncation: Truncation,  
4 ) -> DslPlanDomain:  
5     match truncation:  
6         case Truncation.Filter(_):  
7             for m in domain.margins:  
8                 # After filtering you no longer know partition lengths or keys.  
9                 m.invariant = None #  
10                # Upper bounds on the number of rows and groups remain valid.  
11                return domain  
12         case Truncation.GroupBy(keys, aggs):  
13             for agg in aggs:  
14                 # True means resize is allowed  
15                 check_infallible(agg, True) #  
16  
17             def with_truncation(lf):  
18                 return lf.group_by(keys).agg(aggs)  
19  
20             def without_invariant(m):  
21                 m.invariant = None  
22                 return m  
23  
24             # derive new output domain  
25             return FrameDomain.new_with_margins(  

```

¹See new changes with `git diff f5bb719..f628349 rust/src/transformations/make_stable_lazyframe/truncate/mod.rs`

```

26         [ #
27             Seriesdomain.new_from_field(f)
28             for f in domain.simulate_schema(with_truncation)
29         ],
30         margins=[
31             # discard invariants as multiverses are mixed
32             without_invariant(m.clone())
33             for m in domain.margins
34             # only keep margins that are a subset of the truncation keys
35             if m.by.is_subset(HashSet.from_iter(keys))
36         ],
37     )

```

Postcondition

Theorem 1.1 (Postcondition). Define `function` as the function that takes in a dataset `arg` and returns a dataset with `truncation` applied to it. `truncation` is either a filter or groupby operation.

- If `truncation` is a filter, then `function` returns a subset of rows in the data.
- If `truncation` is a groupby operation, then `function` groups by `truncation.keys` and the identifier, and aggregates by `truncation.aggs`.

For every element x in `domain`, `function(x)` is in the returned domain or raises a data-independent runtime exception.

Proof. First consider the case where `truncation` is a filter. Since filtering is a contractive mapping, the output domain is a subset of the input domain. However, any invariants on the group length and keys are no longer valid. This is reflected in line 9.

Now consider the case where `truncation` is a groupby operation. Since each group becomes one row in the resulting dataset, an arbitrary black-box function may be applied to each group, so long as errors from the black-box function (the aggregates) are data-independent. Line 15 ensures that the aggregates are infallible.

The schema of members of the output domain is determined by the keys and aggregates of the groupby operation, which is computed on line 26. In this case, only margins that are a subset of the grouping keys remain valid, as any bounds on group lengths are not preserved through the black-box aggregation function. Among preserved margins, invariants are discarded. $\square$